Seminar 05 — Databases, ERD & Drizzle

PB138 — Basics of Web Development

"Your data deserves better than an in-memory array that dies on every restart"

Agenda

1. ERD modeling — entities, attributes, relationships
2. Relational databases — PostgreSQL + Docker
3. SQL — a quick refresher
4. ORM vs Query Builder
5. Drizzle — schema, queries, migrations, transactions
6. Connecting Drizzle to Elysia

ERD Modeling

Designing your data before writing a single line of code

What is an ERD?

Entity-**R**elationship **D**iagram — a visual blueprint of your data model.

- **Entity** = a thing you want to store (Student, Course, Instructor)
- **Attribute** = a property of an entity (name, email, credits)
- **Relationship** = how entities connect to each other

```
┌──────────────┐   enrolls   ┌──────────────┐
│  Student     │  ──────────>│   Course     │
├──────────────┤             ├──────────────┤
│ id           │             │ id           │
│ name         │             │ name         │
│ email        │             │ credits      │
└──────────────┘             └──────────────┘
```

Design first, code second. Changing a schema after launch is painful.

Attributes & Keys

Every entity needs a way to identify each record uniquely.

- **Primary key (PK)** — unique identifier for each row
- **Foreign key (FK)** — reference to a PK in another table
- **Nullable** — can the value be missing? (think: optional fields)

```
┌─ students ──────────────────┐
│ id (PK)      uuid           │
│ firstName    text           │
│ lastName     text           │
│ email        text           │
│ uco          text           │
└─────────────────────────────┘

┌─ courses ────────────────────┐
│ id (PK)      uuid            │
│ name         text            │
│ credits      integer         │
│ semester     text            │
│ instructorId (FK) uuid       │
└──────────────────────────────┘
```

Relationships — Cardinality

How many records on each side?

Type	Example	Implementation
1:1	Student ↔️ Profile	FK on either side (unique)
1:N	Department → Instructors	FK on the "many" side

```
students          enrollments          courses

 id   ◄──────   studentId   ──────►   id
                courseId
                grade
```

The junction table can also hold extra data (grade, enrollment date, etc.).

Serial vs UUID

Two common strategies for primary keys:

	Serial (auto-increment)	UUID
Format	`1, 2, 3, ...`	`550e8400-e29b-41d4-a716-446655440000`
Pros	Small, fast, readable	Globally unique, no collisions
Cons	Predictable, conflicts in distributed systems	Larger (16 bytes), less readable
Use when	Simple apps, single DB	APIs, distributed systems, security

We use UUIDs — better for APIs (no guessing `/users/1`, `/users/2`...) and scale-ready.

Relational Databases

Tables, rows, columns — the data structure that has run the world since 1970

PostgreSQL + Docker

PostgreSQL — the most advanced open-source relational database. Battle-tested, feature-rich.

We run it in Docker — no local installation needed:

```
docker compose up -d
```

That's it. Your database is running on `localhost:5432`.

```yaml
# docker-compose.yml (already in the project)
services:
  db:
    image: postgres:18
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: pb138
    ports:
      - "5432:5432"
```

Connection String

Every database client needs a **connection string** to know where and how to connect:

```
postgresql://postgres:postgres@localhost:5432/pb138
|            |         |          |         |    |
protocol     user     password  host      port database
```

Stored in `.env` — never commit credentials to git:

```
# .env
DATABASE_URL="postgresql://postgres:postgres@localhost:5432/pb138"
```

SQL Refresher

The language your database actually speaks

SQL — The Basics

Before ORMs, there was SQL. And it's still what runs under the hood.

```sql
INSERT INTO students (id, first_name, email, uco)
VALUES ('550e...', 'Alice', 'alice@mail.com', '123456');

SELECT * FROM students WHERE uco = '123456';

UPDATE students SET email = 'new@mail.com' WHERE id = '550e...';

DELETE FROM students WHERE id = '550e...';

SELECT s.first_name, c.name FROM students s
JOIN enrollments e ON e.student_id = s.id
JOIN courses c ON e.course_id = c.id;
```

Works, but writing raw SQL strings in TypeScript = no type safety, no autocompletion, easy to break.

ORM vs Query Builder

Two ways to stop writing raw SQL strings

The Spectrum

```
Raw SQL ———————— Query Builder ———————— ORM
 ↑                    ↑                   ↑
 Full control         Typed SQL           Magic
 No safety            Composable          Heavy abstraction
 e.g. pg driver       e.g. Drizzle        e.g. Prisma, TypeORM
```

ORM (Object-Relational Mapping) — maps DB tables to classes/objects. You work with objects, ORM generates SQL. Can feel like magic — until the magic breaks.

Query Builder — you still think in SQL, but with type-safe, composable functions. Closer to the metal, fewer surprises.

Drizzle = query builder first, with an optional ORM-like relational API. Best of both worlds.

Drizzle

"If you know SQL, you know Drizzle"

What is Drizzle?

- TypeScript-first SQL toolkit
- **Schema in code** — your tables are TypeScript, not YAML or decorators
- **Two APIs**: query builder (SQL-like) + relational queries (ORM-like)
- Supports PostgreSQL, MySQL, SQLite
- Built-in migrations

```
import { drizzle } from "drizzle-orm/node-postgres";

const db = drizzle(process.env.DATABASE_URL);

const users = await db.select().from(students);
// → fully typed Student[]
```

Defining a Schema

Tables are defined as plain TypeScript — this IS your source of truth:

```
import { pgTable, uuid, text, integer, timestamp } from "drizzle-orm/pg-core";

export const students = pgTable("students", {
  id: uuid("id").primaryKey().defaultRandom(),
  firstName: text("first_name").notNull(),
  lastName: text("last_name").notNull(),
  email: text("email").notNull().unique(),
  uco: text("uco").notNull().unique(),
  createdAt: timestamp("created_at").defaultNow(),
});
```

Column names use `snake_case` in DB, `camelCase` in TypeScript — Drizzle maps them automatically.

Relationships in Schema

```
export const courses = pgTable("courses", {
  id: uuid("id").primaryKey().defaultRandom(),
  name: text("name").notNull(),
  credits: integer("credits").notNull(),
  instructorId: uuid("instructor_id").references(() => instructors.id),
});

export const enrollments = pgTable("enrollments", {
  id: uuid("id").primaryKey().defaultRandom(),
  studentId: uuid("student_id")
    .notNull()
    .references(() => students.id),
  courseId: uuid("course_id")
    .notNull()
    .references(() => courses.id),
});
```

`.references()` creates a foreign key constraint — the DB enforces referential integrity for you.

Queries — SQL-like API

Drizzle's query builder mirrors SQL — if you know SQL, you can read this:

```
import { eq, gte, and } from "drizzle-orm";

// SELECT * FROM students
const all = await db.select().from(students);

// SELECT * FROM students WHERE uco = '123456'
const one = await db.select().from(students).where(eq(students.uco, "123456"));

// SELECT * FROM courses WHERE credits >= 4 AND semester = 'spring'
const filtered = await db
  .select()
  .from(courses)
  .where(and(gte(courses.credits, 4), eq(courses.semester, "spring")));
```

Every query returns fully typed results. No `any`, no manual casting.

Queries — Insert, Update, Delete

```
// INSERT — returns the created row(s)
const [newStudent] = await db
  .insert(students)
  .values({
    firstName: "Alice",
    lastName: "Nováková",
    email: "alice@mail.com",
    uco: "123456",
  })
  .returning();

// UPDATE
await db
  .update(students)
  .set({ email: "new@mail.com" })
  .where(eq(students.id, studentId));

// DELETE
await db.delete(students).where(eq(students.id, studentId));
```

`.returning()` gives you back the affected rows — useful for confirming what was created/updated.

Queries — Joins

```
// SQL: SELECT s.first_name, c.name FROM students s
//      JOIN enrollments e ON e.student_id = s.id
//      JOIN courses c ON e.course_id = c.id

const result = await db
  .select({
    studentName: students.firstName,
    courseName: courses.name,
  })
  .from(students)
  .innerJoin(enrollments, eq(enrollments.studentId, students.id))
  .innerJoin(courses, eq(enrollments.courseId, courses.id));
```

Same structure as SQL. Fully typed — `result` is `{ studentName: string, courseName: string }[]`.

Query API — `db.query.*`

Drizzle's second API — closer to Prisma. No manual joins, just declare what you want:

```
// Find all courses — with optional filtering, ordering, limit
const courses = await db.query.courses.findMany({
  where: eq(courses.semester, "spring"),
  orderBy: [asc(courses.name)],
  limit: 10,
});

// Find one student by condition
const student = await db.query.students.findFirst({
  where: eq(students.uco, "123456"),
});

// Load related entities — no manual joins
const coursesWithInstructor = await db.query.courses.findMany({
  with: { instructor: true },
});
// → { id, name, credits, instructor: { id, firstName, ... } }[]
```

SQL-like API = full control. Query API = convenience. Use whichever fits.

SQL vs Drizzle — Side by Side

SQL	Drizzle
`SELECT * FROM students`	`db.select().from(students)`
`WHERE uco = '123'`	`.where(eq(students.uco, '123'))`
`INSERT INTO students (...)`	`db.insert(students).values({...})`
`UPDATE students SET ...`	`db.update(students).set({...})`
`DELETE FROM students`	`db.delete(students)`
`JOIN enrollments ON ...`	`.innerJoin(enrollments, eq(...))`

Same concepts, same structure — but type-safe and composable.

Migrations

Your schema will change over time. **Migrations** track those changes.

```
# 1. Generate a migration from schema changes
bunx drizzle-kit generate

# 2. Apply pending migrations to the database
bunx drizzle-kit migrate

# 3. Open Drizzle Studio — visual DB browser
bunx drizzle-kit studio
```

Drizzle Kit compares your schema code to the DB and generates SQL migration files automatically.

```
drizzle/
├── 0000_initial.sql        ← CREATE TABLE students ...
├── 0001_add_courses.sql    ← CREATE TABLE courses ...
└── 0002_add_enrollments.sql
```

Commit migrations to git — they're your database changelog.

Transactions

Sometimes multiple operations must succeed or fail **together** — that's a transaction.

```
// Enroll a student into multiple courses — all or nothing
await db.transaction(async (tx) => {
  for (const courseId of courseIds) {
    await tx.insert(enrollments).values({ studentId, courseId });
  }
});
```

If any insert fails (e.g. duplicate enrollment), **all** inserts are rolled back.

Without a transaction: student could end up enrolled in 2 out of 3 courses — inconsistent state.

Transactions — Architecture Pattern

Repository = pure data access, receives `db` (or `tx`) as a parameter.

Service = owns the transaction, passes `tx` to repository methods.

```
// repository — knows nothing about transactions
const createEnrollment = async (
  db: Database,
  studentId: string,
  courseId: string,
) => {
  const [row] = await db
    .insert(enrollments)
    .values({ studentId, courseId })
    .returning();
  return row;
};

// service — orchestrates the transaction
const bulkEnroll = async (data: BulkEnroll) => {
  return await db.transaction(async (tx) => {
    for (const courseId of data.courseIds) {
      await enrollmentsRepository.createEnrollment(
        tx,
        data.studentId,
        courseId,
      ); // tx, not db!
    }
  });
};
```

Serializable Transactions

Default isolation level allows **read-then-write** race conditions:

```
Transaction A: reads 29 enrolled (capacity 30) → inserts → 30 ✅
Transaction B: reads 29 enrolled (capacity 30) → inserts → 31 ❌ over capacity!
```

Serializable isolation prevents this — the DB detects conflicts and retries/aborts:

```
await db.transaction(
  async (tx) => {
    const course = await coursesRepository.findById(tx, toCourseId);
    const enrolled = await enrollmentsRepository.countByCourse(tx, toCourseId);
    if (enrolled >= course.capacity) throw new Error("Course is full");

    await enrollmentsRepository.deleteEnrollment(tx, studentId, fromCourseId);
    await enrollmentsRepository.createEnrollment(tx, studentId, toCourseId);
  },
  { isolationLevel: "serializable" },
);
```

Soft Delete

Instead of `DELETE` — mark records as deleted. Data stays recoverable, FK constraints stay intact.

```
// Soft delete — set a timestamp
await db
  .update(courses)
  .set({ deletedAt: new Date() })
  .where(eq(courses.id, courseId));

// Always filter in queries
const active = await db.select().from(courses).where(isNull(courses.deletedAt));
```

Common pattern: add a `deletedAt` column (nullable timestamp) to tables you want to soft-delete.

Checkpoint: Drizzle

- Schema defined in TypeScript — single source of truth
- Query builder mirrors SQL — `select`, `insert`, `update`, `delete`, `join`
- Full type safety — results are typed, no `any`
- Transactions — `db.transaction()` for atomic multi-step operations
- Serializable isolation — prevents race conditions on capacity checks
- Repository accepts `db` / `tx` — service owns transaction logic
- Migrations via `drizzle-kit generate` + `drizzle-kit migrate`
- Drizzle Studio for visual DB browsing

Questions?

Exercise: Connect to a Real Database

Getting Started

```
cd pb138-seminars && git checkout seminar-05-assignment
bun install
docker compose up -d        # start PostgreSQL
bun run db:migrate          # apply migrations
bun run dev
```

URL	What
http://localhost:3000	REST API
http://localhost:3000/api-docs	Scalar

The instructors module is already connected to the database — use it as a reference.

Your job: **complete the schema, connect remaining repositories, and implement transactions (including a serializable one).**

Task 1 — Complete the Schema

File: `apps/server/src/db/schema.ts`

Students and instructors tables are defined. Complete the remaining two:

- **`courses`** — code, name, description, credits, instructorId (FK), semester, year, capacity
- **`enrollments`** — studentId (FK), courseId (FK), enrolledAt, unique constraint on (studentId, courseId)

After editing the schema, regenerate and apply the migration:

```
bun run db:generate
bun run db:migrate
```

Verify: Open Drizzle Studio — all 4 tables should appear.

Task 2 — Students Repository

File: `apps/server/src/modules/students/students.repository.ts`

Replace the stubs with Drizzle queries. Use **instructors.repository.ts** as a reference. Each method receives `db` (database or transaction instance) as the first argument:

Method	Hint
`findAll(db)`	`db.select().from(...)`
`findById(db, id)`	`.where(eq(...))`, return first or undefined
`create(db, data)`	`.insert(...).values(data).returning()`

Verify: Scalar → `POST /students` to create, then `GET /students`

Task 3 — Courses Repository

File: `apps/server/src/modules/courses/courses.repository.ts`

The `findAll` query (LEFT JOIN + count) is provided. Complete:

- **`findAll` filter conditions** — build conditions from `filter` using `eq`, `gte`, `lte`, pass to `.where(and(...))`
- **`findById(db, id)`** — same join pattern, filtered by id
- **`create(db, data)`** — insert + return with `enrolled: 0`

Verify: Scalar → `POST /courses` to create, then `GET /courses?semester=spring`

Task 4 — Seed Script

File: `apps/server/src/db/seed.ts`

Instructors & students are already seeded. Complete the script — add courses and enrollments.

Use the already-created `createdInstructors` and `createdStudents` arrays to reference valid IDs.

```
bun run db:seed
```

Verify: Open Drizzle Studio — all tables should have data.

Task 5 — Bulk Enrollment (Transaction)

Files: `enrollments.repository.ts` + `enrollments.service.ts`

1. **Repository** — implement `createEnrollment(db, studentId, courseId)`.
2. **Service** — implement `bulkEnroll` using `db.transaction()`. Call `createEnrollment(tx, ...)` for each course. If any fails, all are rolled back.

Verify: `POST /students/:id/enroll` with `{ "courseIds": [...] }`, then check Drizzle Studio.

Task 6 — Course Transfer (Serializable Transaction)

Files: `enrollments.repository.ts` + `enrollments.service.ts`

Transfer a student from one course to another — only if the target course has capacity.

1. **Repository** — implement `deleteEnrollment(db, ...)` and `countByCourse(db, ...)`.
2. **Service** — implement `transferEnrollment` using `db.transaction(async (tx) => { ... }, { isolationLevel: 'serializable' })`.

Why serializable? Without it, two concurrent transfers could both read "29/30 enrolled" and both insert — exceeding capacity.

Verify: `POST /students/:id/transfer`, then try transferring to a full course — should fail with 409.

Summary

What We Covered

1. **ERD modeling** — entities, attributes, relationships (1:1, 1:N, M:N)
2. **Primary keys** — serial vs UUID, why we use UUIDs for APIs
3. **PostgreSQL + Docker** — `docker compose up -d` and you're running
4. **SQL refresher** — CRUD operations, joins
5. **ORM vs Query Builder** — Drizzle sits in the sweet spot
6. **Drizzle schema** — tables as TypeScript, foreign keys, constraints
7. **Drizzle queries** — SQL-like API, fully typed results
8. **Transactions** — `db.transaction()` for atomic operations
9. **Serializable transactions** — preventing race conditions on read-then-write
10. **Migrations** — `drizzle-kit generate` + `migrate`

"A database is not just storage. It's the contract your application makes with reality."

Now let's connect to a real database.

Exercises time!